

React + NestJS Intern Assignment

Mini Job Queue Dashboard

Build a small **Job Queue Management Dashboard** using **React.js** for the frontend and **NestJS** for the backend.

The goal is not to build a large application. We want to see how you approach **API design, state management, validation, edge cases, and real-world concurrency problems**.

1. Backend — NestJS

Create APIs to:

- `POST /jobs` — Create a job
- `GET /jobs` — Get all jobs
- `PATCH /jobs/:id/status` — Update job status
- `DELETE /jobs/:id` — Delete a job

Each job should contain:

```
id
title
type
status
createdAt
```

Allowed statuses:

```
pending
running
completed
failed
```

Use **PostgreSQL** or **SQLite** for persistence.

Include appropriate validation and error handling.

2. Frontend — React

Create a simple dashboard that:

- Displays all jobs
- Filters jobs by status
- Creates a new job
- Changes job status
- Deletes a job
- Displays counts for each status
- Handles loading and API error states

Keep the UI simple. We care more about the implementation than visual design.

3. Think About This

A job can only follow these transitions:

```
pending → running → completed
                ↘ failed
```

A `completed` or `failed` job cannot become `running` again.

Important: Imagine two browser tabs are open. Both users see a job as `pending` and both try to change it to `running` at almost the same time.

Think about:

- Where should this rule be enforced?
- What happens if someone bypasses the React application and calls the API directly?
- What happens when two requests arrive at nearly the same time?
- How would you prevent an invalid or inconsistent state?

You don't need to implement an overly complicated distributed system. We mainly want to see **how you identify and reason about the problem**.

4. Submission Requirements

Please submit:

- **GitHub repository link**
 - Keep the repository public.
 - Include a clear `README.md` with setup instructions and any important decisions you made.
- **Live frontend URL**
- **Live backend/API URL**
 - Briefly mention any assumptions, trade-offs, or additional improvements you would make with more time.

Both the frontend and backend should be **deployed and accessible through the submitted links**.

5. Time Expectation

The Deadline would be 2 days since you receive the assignment.

Do not spend excessive time on UI polish. We are primarily evaluating:

- React fundamentals
- NestJS fundamentals
- API and database design

- Validation and error handling
- Code quality
- Problem-solving
- Critical thinking about edge cases

Bonus

Add one small improvement that you think would make this system more production-ready.

Explain why you chose it in the README.

There is no single "correct" implementation. We are interested in **your reasoning and engineering decisions**.